Team 17 — Step-by-step build guide

Oct 28, 2026 · @sempermanager

This guide builds the whole system one part at a time. After each part you run a small script, so a problem shows up straight away and you always know which part caused it. All files are in the `Team17_Pico` folder on GitHub; the full wiring is in `Team17_Wiring_reference.png`.

Before you start

What you need on the table

- Pico on the MonkMakes Pico breadboard, micro-USB cable, laptop with Thonny
- LEDs (green, yellow, red), 3 × 330 Ω, 2 × 10 kΩ, 1 × 4.7 kΩ, LDR, passive buzzer, push button (kit)
- Relay HW-482, 12 V solenoid, diode 1N4007
- 12 V adapter (GA-1201200V), green DC-jack screw terminal, XL4015 step-down board
- Raspberry Pi 4 + its power supply, DS3231 clock, Ethernet cable
- ESP8266 HUZZAH on its own small breadboard
- Jumper wires (male–male, male–female, female–female), a little hook-up wire, a small screwdriver, a multimeter (lab)

Three rules, every time

1. Unplug the Pico's USB and the 12 V adapter before you move any wire.
2. 5 V or 12 V never goes into a Pico row on the breadboard. It can kill the Pico.
3. Check the new wires against the picture before you plug back in.

Thonny, once

1. Hold the BOOTSEL button on the Pico, plug in the USB, let go.
2. In Thonny, click the bottom-right corner → *Configure interpreter* → *MicroPython (Raspberry Pi Pico)* → *Install or update MicroPython*.
3. To run a script: open it and press **Run** (F5). It runs from the laptop; nothing is saved on the Pico. **Stop** ends it.

How holes are named

- Each hole = column letter + row. The Pico sits in columns c and h, USB at the top.
- Beside the Pico, the board prints pin labels (TX, RX, G, 2, 3 … on the left; 5V, Vs, G, 3e, 3V … 16 on the right) instead of row numbers. We write those holes with the label in brackets: `a[2]` = column a, row printed **2** (GP2). `j[3V]` = column j, row printed **3V**.

- The board prints **G** (ground) eight times, so we say where: `a[G 9/10]` = the G between 9 and 10. The G between 28 and 27 on the right is **AGND**, only for the light sensor.
- Below the Pico, rows are numbered 21–30: `a26` = column a, row 26. Careful: `j[26]` (GP26, beside the Pico) and `j26` (free row 26) are different holes.
- In each row, a–e are joined and f–j are joined. The two halves are not joined. The + / − rails on the sides are not used.

Step 1 — The Pico on its own

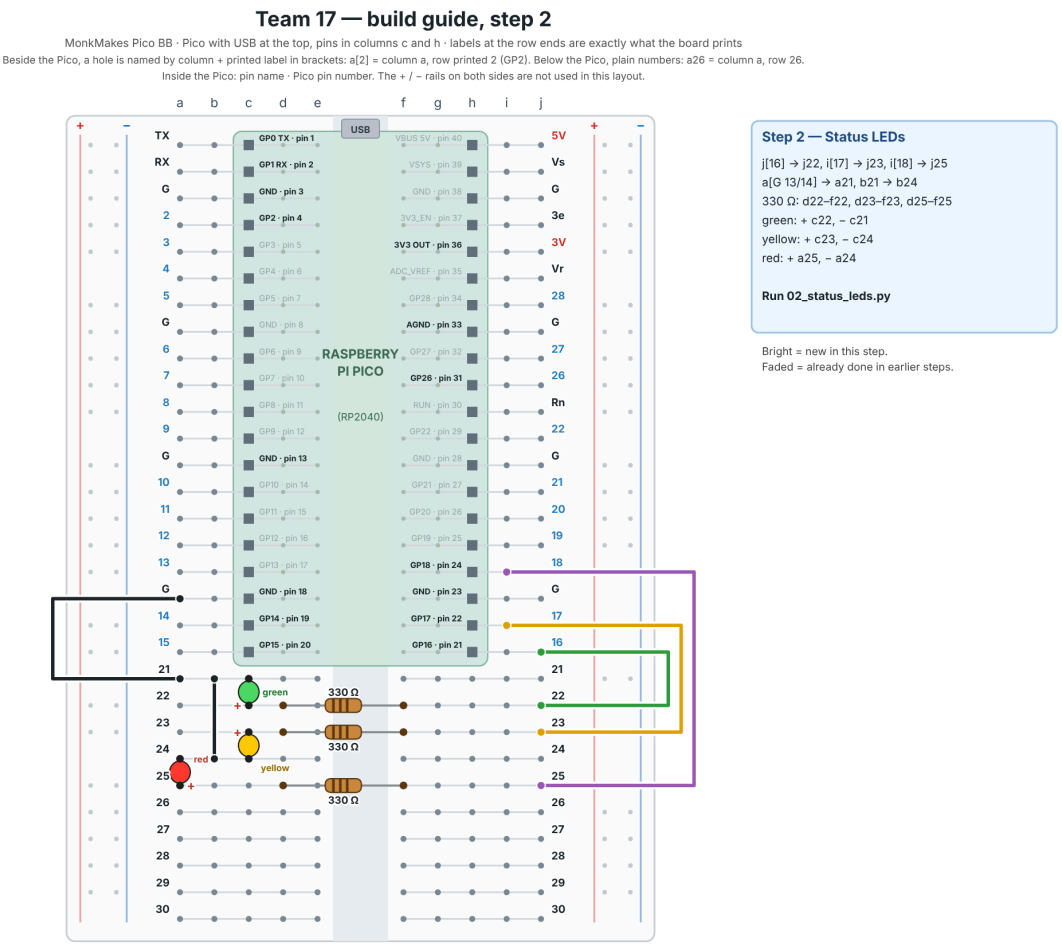
Goal: prove the Pico, the cable and Thonny work. No wiring.

1. Pico on the breadboard, USB to the laptop.
2. Open `01_onboard_blink.py`, press **Run**.
3. **You should see:** the small LED on the Pico blinks 5 times, and the shell prints `Blink test finished`.

Nothing happens? Check the bottom-right corner of Thonny says *MicroPython (Raspberry Pi Pico)* and try another USB cable — some cables only charge.

Step 2 — Status LEDs

Goal: three LEDs the Pico can switch on: green = OK, yellow = starting or network down, red = alarm.



What	From	To
Wire (GP16, green)	j[16]	j22
Wire (GP17, yellow)	i[17]	j23
Wire (GP18, red)	i[18]	j25
Wire (ground)	a[G 13/14]	a21
Short wire (ground, row 21 to row 24)	b21	b24
330 Ω	d22	f22
330 Ω	d23	f23
330 Ω	d25	f25
Green LED	long leg **c22**	short leg c21
Yellow LED	long leg **c23**	short leg c24
Red LED	long leg **a25**	short leg a24

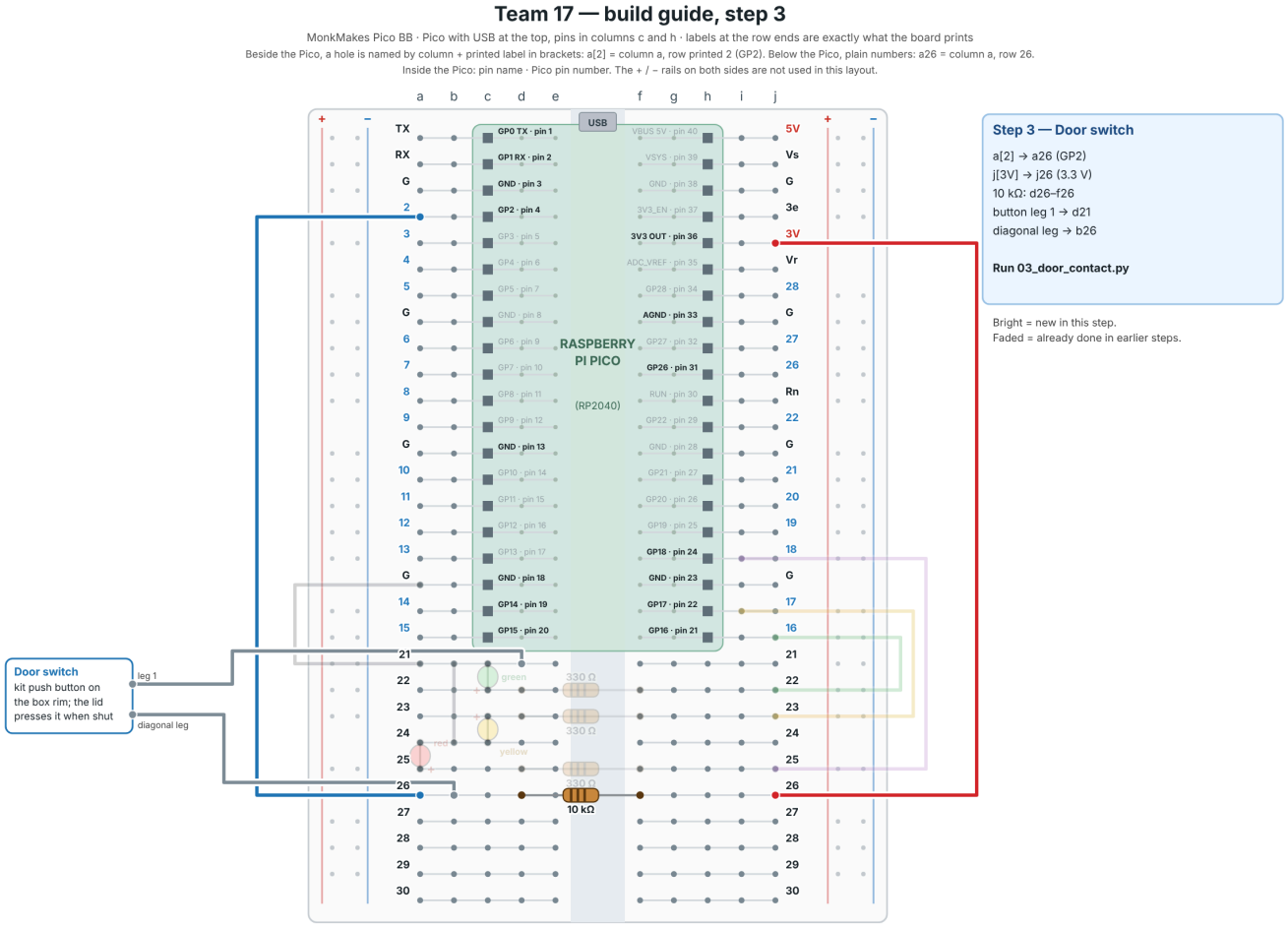
The three resistors lie across the middle channel. The long leg of an LED is +.

Run `02_status_leds.py`. **You should see:** green, then yellow, then red, 1 second each, then all three together.

One LED stays dark? Turn it round (long leg on the resistor side) and check its short leg is in row 21 or 24.

Step 3 — Door switch (push button)

Goal: the Pico knows if the box is shut, open, or if someone cut the wire.



What	From	To
Wire (GP2)	a[2]	a26
Wire (3.3 V)	j[3V]	j26
10 kΩ (pull-up)	d26	f26
Male–female wire	d21 (ground)	button, leg 1
Male–female wire	b26	button, the leg **diagonally opposite** leg 1

- For now the button can sit loose on the table. In step 14 it goes on the rim of the box, so the lid presses it when it closes.
- Use two **diagonal** legs: a 4-leg button joins its legs in pairs inside, and diagonal legs are always on opposite sides of the switch.
- Female ends loose on the legs? A bit of tape or heat-shrink holds them.

Run `03_door_contact.py` (it keeps running until you press Stop).

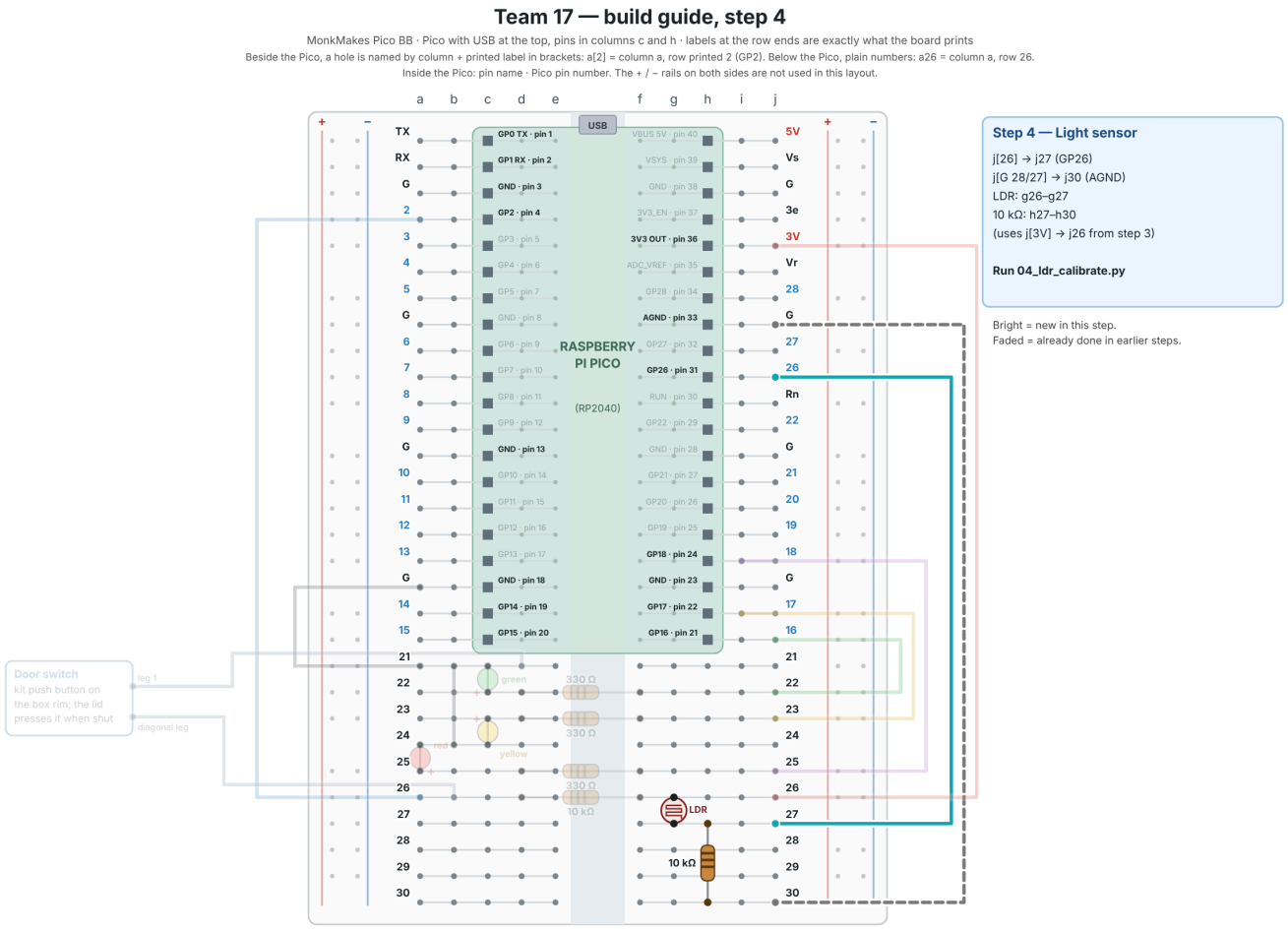
You should see:

- Button pressed → `Door: SHUT`
- Button released → `Door: OPEN (or wire cut)`
- Pull one button wire out → `OPEN (or wire cut)` too. That is on purpose: a cut cable must look like an alarm.

Always SHUT, even released? The two wires are on legs joined inside the button — move one to the diagonal leg.

Step 4 — Light sensor (LDR)

Goal: the Pico measures the light inside the box. Opening the lid lets light in, so this is the second way to notice the box being opened.



What	From	To
Wire (GP26)	j[26]	j27
Wire (AGND)	j[G 28/27]	j30
LDR	g26	g27
10 kΩ (load)	h27	h30

It uses the 3.3 V wire from step 3 (j[3V] → j26). The LDR has no + or −. Row 30 **right** is AGND — use it only for the light sensor.

Run `04_ldr_calibrate.py`.

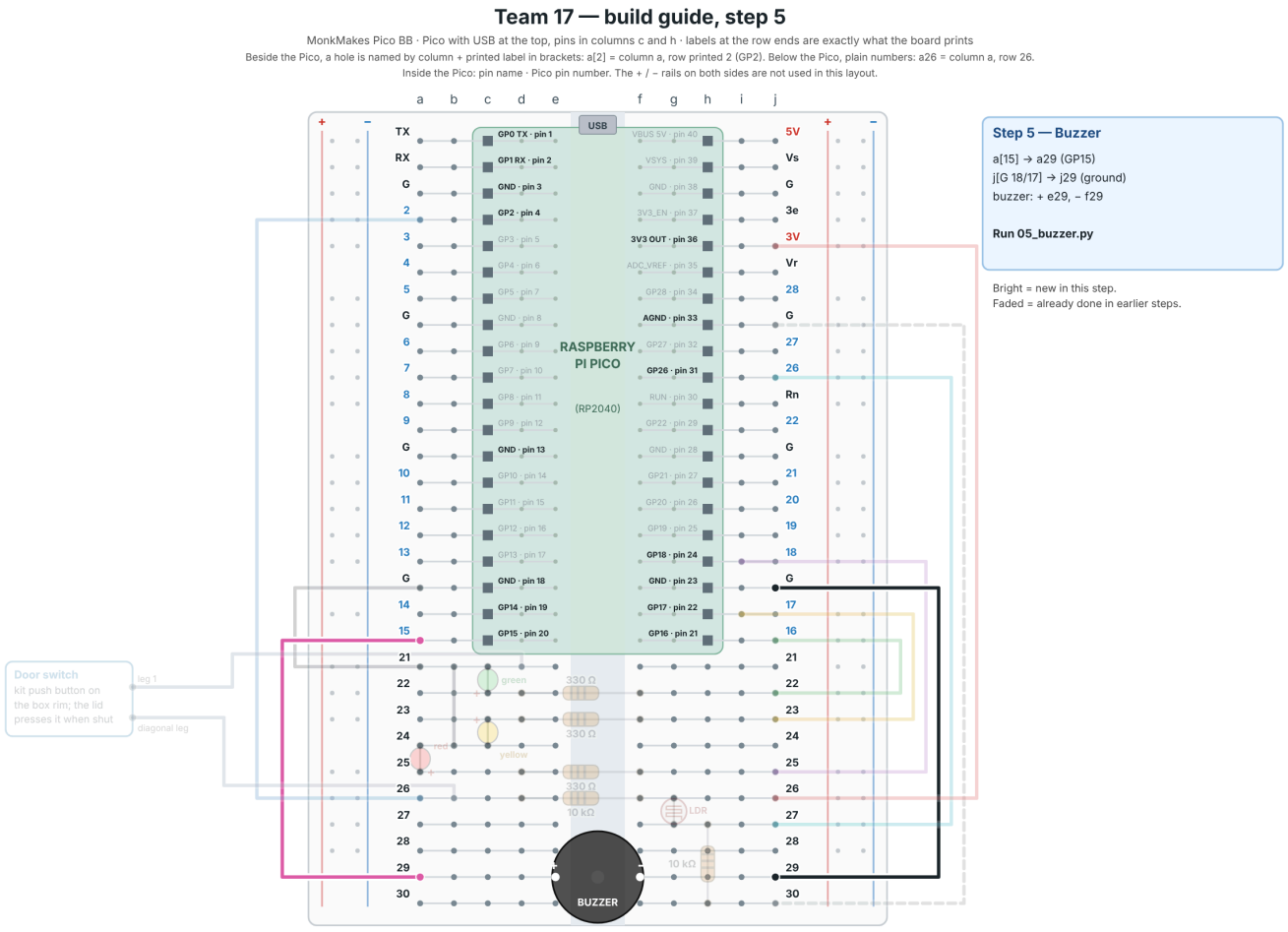
You should see: 5 seconds of live readings (`light: 35 %` and so on). Cover the LDR with your hand: the number drops. Then it asks you to make it dark and press Enter, then light and press Enter, and prints `Suggested LIGHT_THRESHOLD_PCT = ...`.

- For now, "dark" = your hand over the LDR. In step 14 you do it again inside the real box, and that number is the one you keep.

- Write the number into `LIGHT_THRESHOLD_PCT` at the top of `07_tamper_test.py` and `08_main.py`.
- If it says the gap is small, swap the 10 kΩ in h27–h30 for the 4.7 kΩ and run it again.

Step 5 — Buzzer

Goal: a local alarm sound.



What	From	To
Wire (GP15)	a[15]	a29
Wire (ground)	j[G 18/17]	j29
Buzzer	**+ leg e29**	– leg f29

The buzzer sits across the middle channel. The + is marked on top (or it is the longer leg). Its round body covers the middle holes of rows 27–30 — that is why nothing else goes there.

Run `05_buzzer.py`.

You should see / hear: three short beeps, then `Buzzer test finished`.

Silent? Check the + leg is in e29. It is a passive buzzer: it needs a tone, which the script makes — so a plain on/off test won't work.

Step 6 — Tamper test (no new wiring)

Goal: check that door switch, light sensor, LEDs and buzzer work together.

1. Put your number from step 4 in `LIGHT_THRESHOLD_PCT` at the top of `07_tamper_test.py`.
2. **Run** `07_tamper_test.py` (runs until Stop).

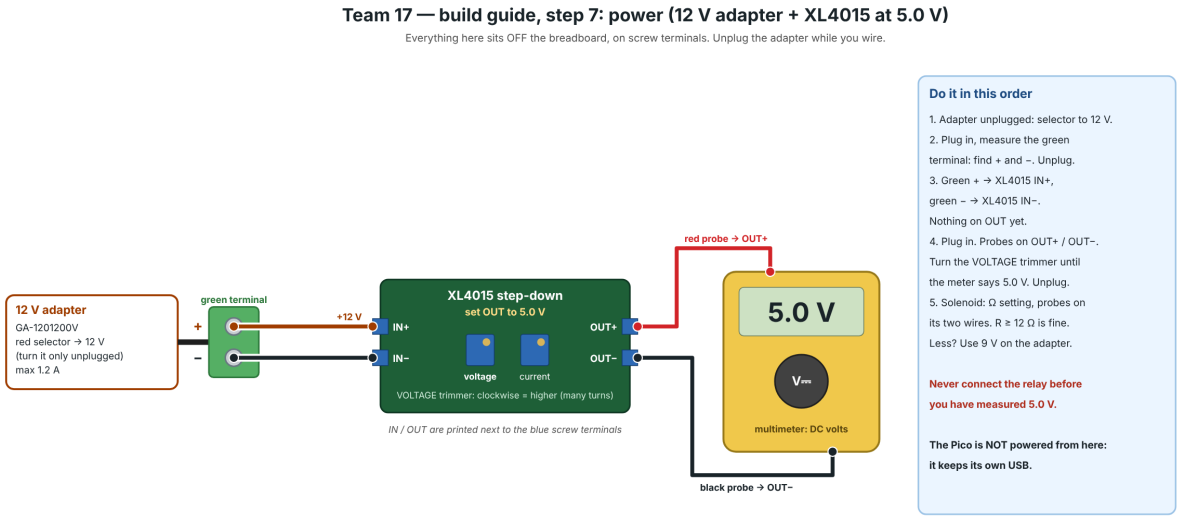
You should see:

You do (in this order)	LED	Buzzer	Shell
Hold the button down, hand over the LDR	green	quiet	`Clear | light ... %`
Let go of the button	red	on	`ALARM: DOOR`
Press the button again (LDR still covered)	green	quiet	`Clear`
Keep it pressed, take your hand off the LDR	red	on	`ALARM: LIGHT`
Cover the LDR again	green	quiet	`Clear`

When this works, the whole tamper side is done.

Step 7 — Power: 12 V adapter and XL4015 at 5.0 V

Goal: one adapter gives 12 V for the lock, and the XL4015 turns it into 5 V for the relay. Nothing is connected to the breadboard in this step. You need the multimeter.



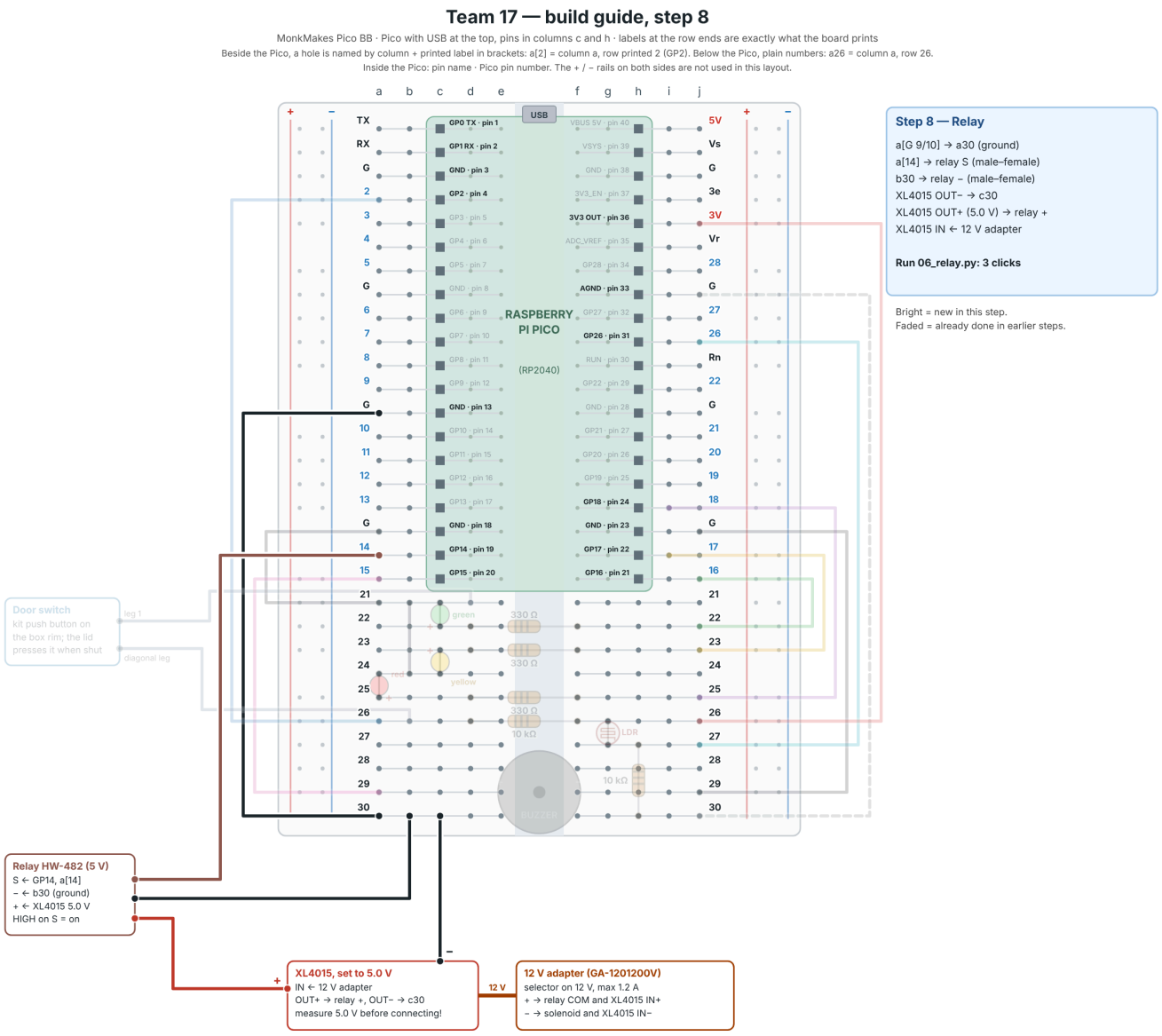
1. Adapter **unplugged**: turn the red selector to **12 V**. Only ever turn it while unplugged.
2. Plug the adapter into the green DC-jack screw terminal, plug it into the wall, and measure the two screws (DC volts). Note which screw is **+**. Unplug.
3. Find **IN** and **OUT** on the XL4015 — the labels are printed next to its two blue screw terminals.
4. Wire green **+** → XL4015 **IN+**, green **–** → XL4015 **IN–**. Nothing on the output. Check + and – twice: the XL4015 has no protection against them being swapped.
5. Plug in. Measure **OUT+ / OUT–**. Turn the **voltage** trimmer (small blue box with a brass screw, usually the one nearer the big XL4015 chip; clockwise = higher; many turns) until it reads **5.0 V**. If the number doesn't move, you have the other trimmer. Unplug.
6. Measure the solenoid's resistance (Ω setting, probes on its two wires). Current = 12 ÷ R. **R of 12 Ω or more** is fine. If it is lower, set the adapter to **9 V** instead — the adapter gives at most 1.2 A.

Never connect anything to the XL4015 output before you have measured 5.0 V on it. It can give much more, and that would burn the relay.

The Pico does not use this power. It stays on its own USB (the laptop now, a USB charger later).

Step 8 — Relay (HW-482)

Goal: the Pico can click the relay. The solenoid is not connected yet.



The relay has 3 pins: **S**, **+**, **−** (read the letters next to the pins — the order can differ).

What	From	To
Wire (ground)	a[G 9/10]	a30
Male–female wire (GP14)	a[14]	relay **S**
Male–female wire (ground)	b30	relay **−**
XL4015 **OUT−**	c30	—
XL4015 **OUT+** (5.0 V)	relay **+** (female end of a jumper)	—

- Row 30 left is where the grounds meet: Pico (a30), relay (b30), XL4015 (c30). This relay has no isolation inside, so without this shared ground it won't react to the Pico.

- XL4015 OUT+ goes straight to the relay. It never touches the breadboard.
- Leads with small white plugs (for OUT1–OUT4) are usually red = +, black = −. Measure once to be sure.

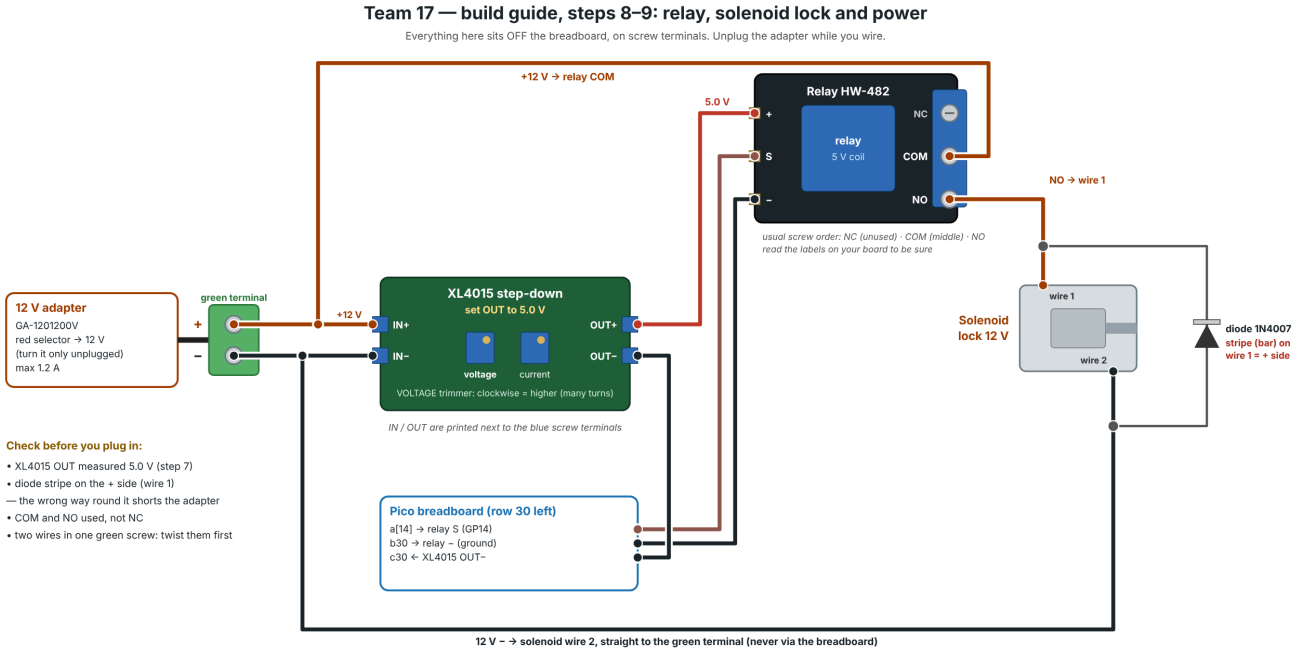
Plug in the 12 V adapter, then the Pico. **Run** `06_relay.py`.

You should see / hear: three clicks, the little LED on the relay going on and off, and `Relay ON (lock open)` / `Relay OFF (lock closed)` in the shell.

No click? Measure 5.0 V on the relay's + and − pins. Check the shared ground in row 30 and the S wire from a[14].

Step 9 — Solenoid lock and diode

Goal: the relay moves the lock. Everything here is on screw terminals, away from the breadboard. Unplug the adapter first.



What	From	To
12 V **+** (green terminal)	relay **COM**	also XL4015 **IN+** (already there)
12 V **−** (green terminal)	solenoid wire 2	also XL4015 **IN−** (already there)
Relay **NO**	solenoid wire 1	—
Diode 1N4007	**stripe end** on solenoid wire 1 (+ side)	other end on solenoid wire 2

- Two wires in one screw of the green terminal: twist them together first.
- **COM** and **NO** = "normally open": no command, or no power on the relay, means no current to the solenoid. The usual order is NC · COM (middle) · NO, but read the labels next to the three screws.
- The diode goes "backwards" across the solenoid. When the relay lets go, the solenoid kicks back a high voltage and the diode soaks it up. **Check the stripe before you plug in:** the wrong way round, it shorts the adapter.
- Take the solenoid's − straight to the green terminal, not through the breadboard, so the solenoid's current never runs through the Pico's ground wires.

Plug in the adapter, then the Pico. **Run** `06_relay.py` again.

You should see: the lock pin moves in and out three times with the clicks (1 second each).

The relay clicks but the lock doesn't move? Adapter on 12 V? COM and NO, not NC? Relay clicks by itself or the Pico resets when the lock moves? The solenoid wants more than 1.2 A — try 9 V.

Step 10 — The full controller, tested from the laptop

Goal: the real Pico program (`08_main.py`) runs on its own, and the laptop plays the part of the Pi with `09_link_tester.py` . No new wiring.

Put the program on the Pico

1. Open `08_main.py` . At the top, check `LINK = "USB"` and put your number in `LIGHT_THRESHOLD_PCT` .
2. *File → Save as → Raspberry Pi Pico →* name it **`main.py`** . From now on it starts by itself whenever the Pico gets power.
3. Unplug and replug the Pico. **Close Thonny** — it holds the USB port.

Start the tester on the laptop

1. Once: `pip install pyserial`
2. Find the Pico's port: Windows `COM3`, `COM4` … (Thonny showed it bottom-right); Mac `/dev/tty.usbmodem…`; Linux `/dev/ttyACM0`.
3. `python 09_link_tester.py COM4` (your port).

Tape the button down and keep the LDR dark while you test, otherwise the Pico (correctly) shows a door or light alarm.

Type these and check:

Type	You should see
(nothing, just wait)	A status line every second: `DOOR=0 LIGHT=0.. RELAY=0 STATE=OK`; green LED
`on`	`ANSWER: OK`, the lock opens, and closes by itself 3 s later (`RELAY=1` then back to `0`)
`bad`	`ANSWER: REJECT,CHECKSUM` — the lock does not move
`value`	`ANSWER: REJECT,VALUE`
`unknown`	`ANSWER: REJECT,UNKNOWN`
`long`	`ANSWER: REJECT,TOO_LONG`
`junk`	`ANSWER: REJECT,FORMAT`
`netdown`	`STATE=NETDOWN`, yellow LED
`netup`	back to `STATE=OK`, green
`stop`	after about 5 s: `ALARM: LINK`, red LED, buzzer, relay off (SAFE state)
`on` (while stopped)	`ANSWER: REJECT,STATE` — no unlocking while the link is lost
`start`	back to green
release the button	`ALARM: DOOR`, red LED, buzzer
`quit`	prints how many messages were damaged or missing (should be 0)

These are tests T1, T4, T5, T6 and T10 from the proposal — take a screenshot of each for the report.

To change the program later: open Thonny, press **Stop** (it interrupts `main.py`), edit, save again as `main.py`.

Step 11 — Raspberry Pi and the clock (DS3231)

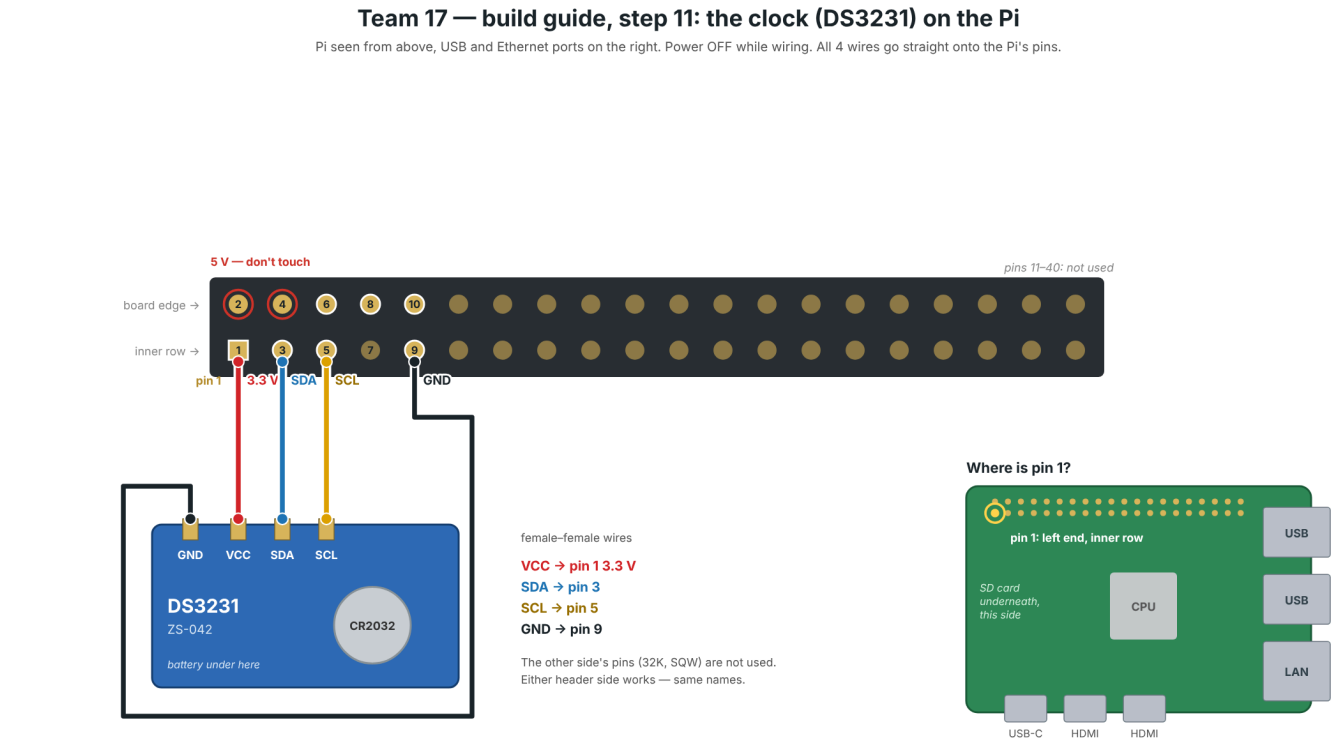
Goal: the Pi runs without screen or keyboard, its serial port is free for the Pico, and it keeps the right time even with no network.

Set up the Pi (once)

1. Raspberry Pi Imager → *Raspberry Pi OS Lite (64-bit)*. In its settings: hostname `gateway-01`, a username that is **not** `pi`, SSH on with **public key only** (paste your laptop's key).
2. Ethernet in, power on, then from the laptop: `ssh <user>@gateway-01.local`. No password should be asked.
3. `sudo apt update && sudo apt full-upgrade`, then `sudo reboot`.
4. `sudo raspi-config` → *Interface Options* → *Serial Port*: login shell **No**, hardware **Yes**. Then *Interface Options* → *I2C* → **Yes**.
5. Add `dtoverlay=disable-bt` to `/boot/firmware/config.txt`, run sudo systemctl disable hciuart (stops the Bluetooth service from grabbing the port), then reboot. Check: `ls -l /dev/serial0` points to `ttyAMA0`.
6. `sudo usermod -aG dialout $USER`, then log out and in (so your user can open the serial port).

Wire the clock — Pi switched off

Four female–female wires, straight onto the Pi's pins:



Check before power-on

- Nothing on pins 2 or 4 (5 V). The clock's VCC is on pin 1 (3.3 V) — on 5 V this module tries to charge its battery.
- After boot: sudo i2cdetect -y 1 shows 68 (the clock) and usually 57 (memory on the same module).

DS3231	Pi pin
VCC	**1** (3.3 V)
SDA	**3**
SCL	**5**
GND	**9**

Pin 1 is at the end of the header furthest from the USB/Ethernet ports, in the inner row. Never pins 2 or 4 (5 V): on 5 V this clock module tries to charge its coin cell. Check there is a CR2032 in its holder.

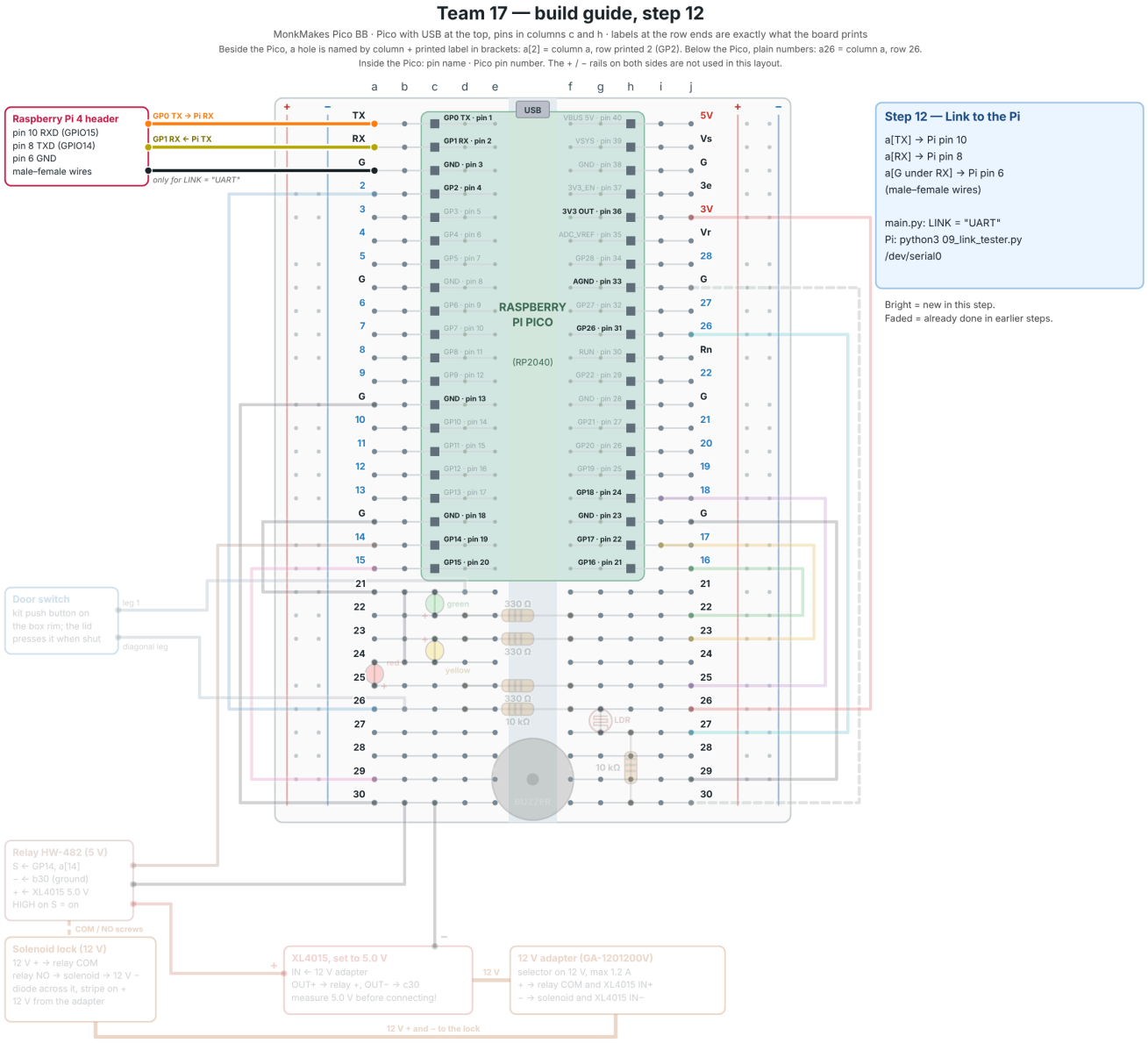
Check it

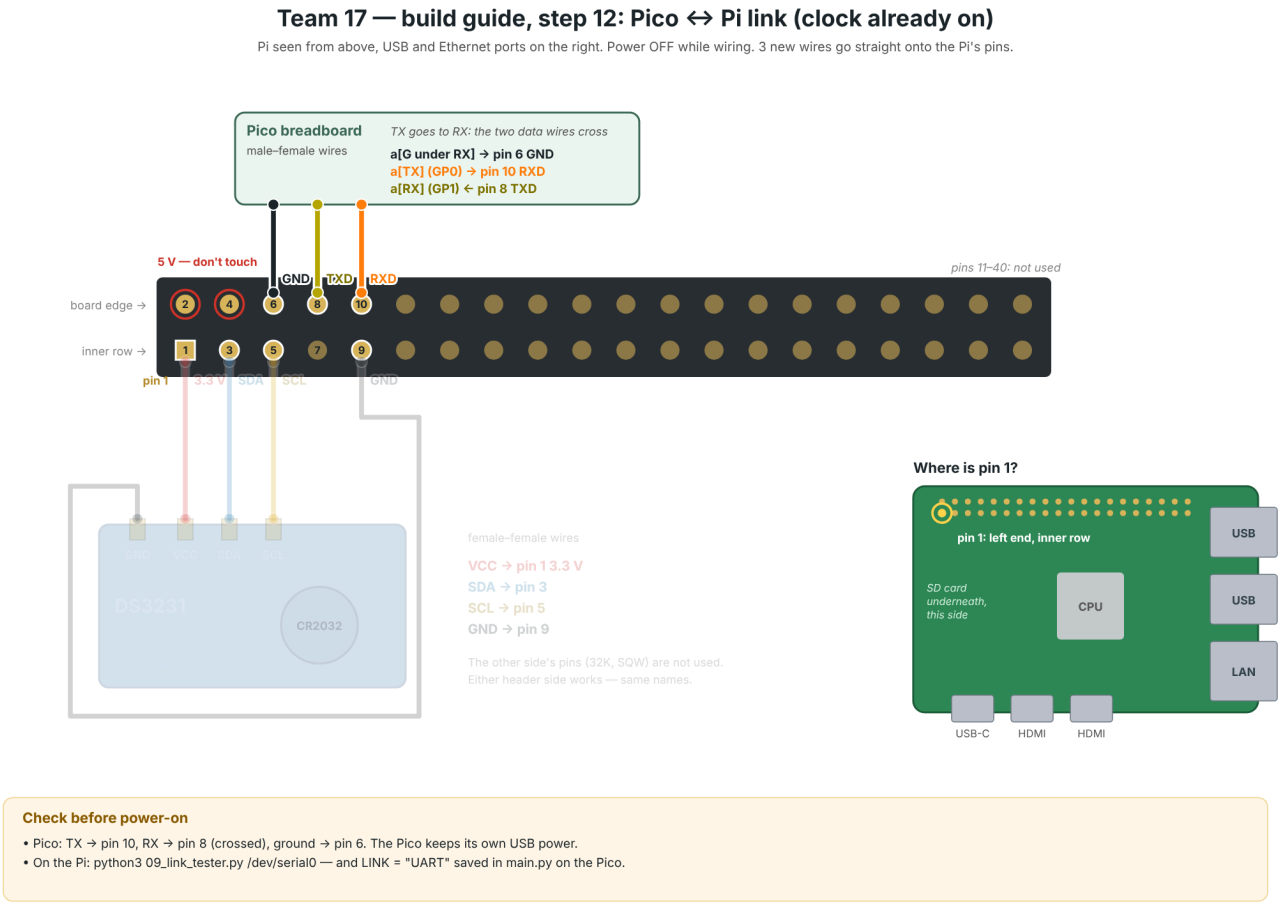
1. Power on. `sudo i2cdetect -y 1` → you should see **68** (the clock) and usually **57** (a small memory chip on the same module — normal).
2. Add `dtoverlay=i2c-rtc,ds3231` to `/boot/firmware/config.txt`, reboot. Now `i2cdetect` shows **UU** at 68: that means Linux is using the clock. Good. dmesg | grep -i rtc should also mention rtc0.

3. With the network on (so the time is right): `sudo hwclock -w` stores the time in the clock; `sudo hwclock -r` reads it back.
4. Test: unplug the network, reboot, run `date` — the time should still be correct.

Step 12 — Pico talks to the Pi

Goal: the same tests as step 10, but now the Pi sends the heartbeat over three wires instead of the laptop over USB.





Wire it — Pi off, Pico unplugged (three male–female wires)

Pico hole	Pi pin
a[TX] (GP0)	**10** (RXD)
a[RX] (GP1)	**8** (TXD)
a[G under RX]	**6** (GND)

TX goes to RX — the two data wires cross. Both sides are 3.3 V, so no converter is needed. The Pico keeps its own USB power; only these three wires join the two boards.

Switch the Pico to the wires

1. Plug the Pico into the laptop, open Thonny, press **Stop**.
2. Open `main.py` on the Pico, change to `LINK = "UART"`, save it (still as `main.py`).

Run the tester on the Pi

1. Copy the tester over: `scp 09_link_tester.py <user>@gateway-01.local:~`
2. On the Pi: `sudo apt install python3-serial`

3. Power the Pico (USB charger or laptop), then on the Pi: `python3 09_link_tester.py /dev/serial0`

You should see: the same results as the table in step 10. Extra test (T5): pull the wire from Pi pin 8 → within about 5 s the Pico goes red, beeps and locks; put it back → green again.

Nothing arrives? TX and RX crossed? Ground wire on pin 6? Serial console switched off (step 11)? `LINK = "UART"` really saved?

Then the real gateway. `gateway/gateway.py` takes over from the tester. It must send exactly the messages in `PROTOCOL.md`. The version on GitHub (23 Oct) sends `$HB*5C`, which the Pico rejects (the heartbeat needs a counter and `NET=OK`, and its checksum would be `0A`, not `5C`). Fix that first, or the Pico goes to SAFE after 5 seconds.

Step 13 — ESP8266 health node (no wiring)

Goal: the HUZZAH reports its own health to the Pi over Wi-Fi every 10 s. It has **no wire** to the Pico or the Pi, on purpose: it is the separate, non-critical Wi-Fi zone.

Needs first: mosquitto running on the Pi (gateway Phase 8), and a Wi-Fi with a normal password that the Pi is on too — for example a phone hotspot. The ESP8266 can't join eduroam.

1. HUZZAH on its own breadboard, USB to the laptop.
2. Thonny: bottom-right → *Configure interpreter* → *MicroPython (ESP8266)* → *Install or update MicroPython*, pick the HUZZAH's port.
3. Copy `node_config_example.py` to `node_config.py`. Fill in the Wi-Fi name, password and the Pi's **IP address** (not `gateway-01.local` — the ESP8266 can't look that up). *Save as* → *MicroPython device*. Never put this file on GitHub.
4. Open `10_health_node_esp8266.py`, *Save as* → *MicroPython device* → `main.py`. Press the HUZZAH's reset button.
5. On the Pi: `mosquitto_sub -t 'site/node-01/health' -v`

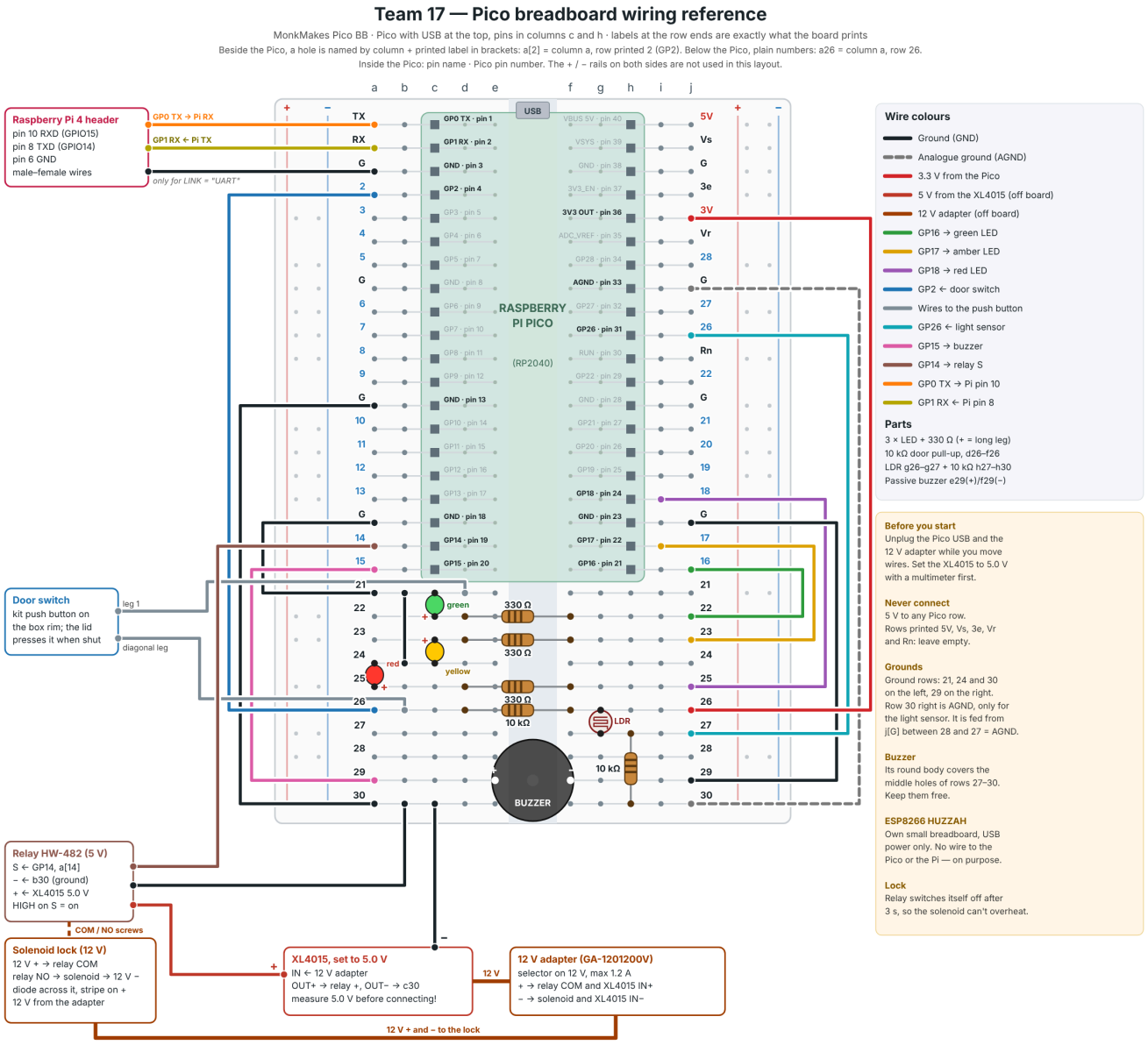
You should see: a line every 10 s like `{"node": "node-01", "online": true, "uptime_s": 40, "rssi": -61, "free_heap": 28000, "reconnects": 0}`, and the blue LED near the antenna blinks once per message.

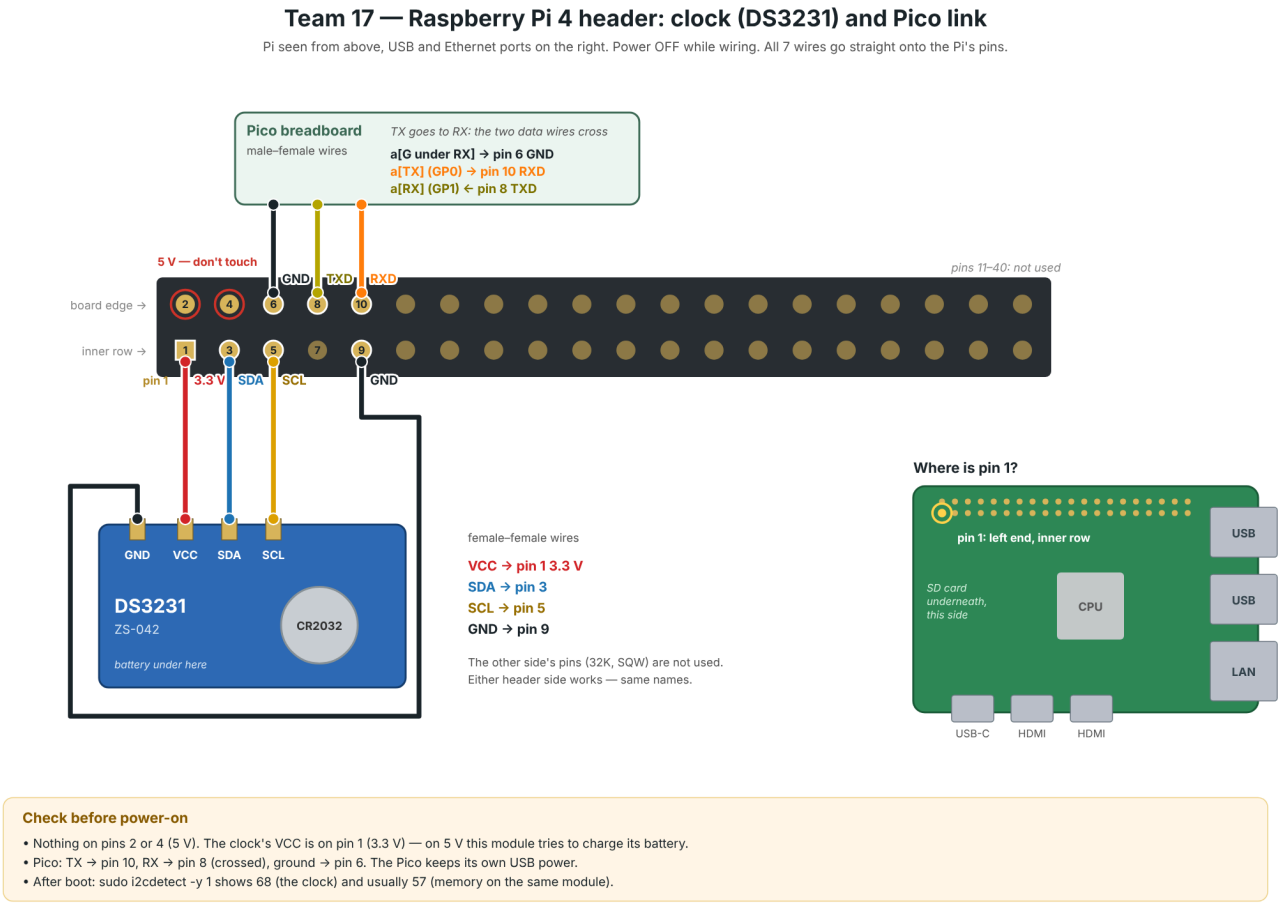
Test: switch the hotspot off for 20 s and on again → it comes back by itself and `reconnects` goes up by 1. Unplug the HUZZAH → after about 45 s the broker posts `{"online":false}` for it.

`no module named 'umqtt'`? Thonny: *Tools* → *Manage packages* → install `micropython-umqtt.simple` onto the board.

Step 14 — Into the box, and the final check

Goal: everything inside the box, tuned to the real box, then the full test once more.





Check before power-on

- Nothing on pins 2 or 4 (5 V). The clock's VCC is on pin 1 (3.3 V) — on 5 V this module tries to charge its battery.
- Pico: TX → pin 10, RX → pin 8 (crossed), ground → pin 6. The Pico keeps its own USB power.
- After boot: sudo i2cdetect -y 1 shows 68 (the clock) and usually 57 (memory on the same module).

Our box has a transparent lid. Make it dark first, or the light sensor sees almost no difference between shut and open. Tape black card (or a few layers of black electrical tape) on the **inside** of the lid. Card held with a few strips of tape comes off again, so the video can still show the inside. Want the status LEDs visible from outside? Push them through 5 mm holes in the card, so each LED fills its hole.

1. **Placement.** Pi, Pico breadboard, relay and XL4015 inside. The push button on the rim, where the lid presses it. The solenoid where its pin can hold the lid.
2. **Keep the LDR away from the LEDs**, or put a bit of black tape or tubing around its sides. Otherwise the red alarm LED itself lights the LDR and can keep the light alarm on after the lid is shut.
3. **Cables in:** Pi power, Pico power, 12 V adapter, Ethernet. Seal gaps with tape — less light leaks in.
4. **Calibrate again, in the real box.** Plug the Pico into the laptop, press Stop in Thonny, run `04_ldr_calibrate.py` with the lid shut (LEDs on, as in normal use), then open. Put the new number in `LIGHT_THRESHOLD_PCT` in `main.py`.
5. **Watchdog on.** In `main.py` set `USE_WATCHDOG = True` and save. From now on, if the program ever hangs, the Pico restarts itself within 2 s. (While editing, Thonny's Stop also triggers a restart — press Stop a few times quickly to get control.)

6. **Compare every wire** with `Team17_Wiring_reference.png` and `Team17_Pi_header.png`.
7. **Final run** of the step 12 tests with the lid shut, plus: open the lid → `ALARM: DOOR` and `ALARM: LIGHT`; cut the power to everything and back on → it all comes back by itself, Pico with `CAUSE=POWER` in its first message.

Record each test (screenshot, log lines or a short clip) for the report and the 90-second video.

If something doesn't work

Problem	Check
LED stays dark	Turned round? Long leg on the resistor side, short leg in row 21 or 24.
Door always OPEN	10 kΩ in d26–f26? 3.3 V wire j[3V]–j26? Button wires in d21 and b26 on **diagonal** legs?
Door always SHUT	Both wires on legs joined inside the button — move one to the diagonal leg.
Light stuck near 0 or 100	LDR in g26–g27? 10 kΩ in h27–h30? AGND wire j[G 28/27]–j30? GP26 wire from j[26] (beside the Pico), not j26?
Buzzer silent	+ leg in e29?
Relay doesn't click	5.0 V on the relay's + and − (measure)? Grounds meet in row 30 left? S wire from a[14]?
Relay clicks, lock doesn't move	Adapter on 12 V? COM and NO, not NC? Diode stripe on the + side?
Pico resets or relay chatters when the lock moves	Solenoid wants more than 1.2 A → adapter to 9 V. Solenoid − straight to the green terminal, not via the breadboard?
Tester can't open the port	Thonny still open? Wrong port name? On the Pi: `dialout` group, serial console off?
Nothing over the Pi wires	TX/RX crossed? Ground on pin 6? `LINK = "UART"` saved in `main.py`?

Problem	Check
Red + beeping straight after power-on	Correct if the lid is open or the button isn't pressed — an open circuit means "door open". Or no heartbeat yet from the Pi.
Light alarm won't clear with the lid shut	The LEDs are lighting the LDR — shield it, then calibrate again (step 14).
`i2cdetect` shows nothing	Clock wires on pins 1, 3, 5, 9? I2C switched on in `raspi-config`?
Time wrong after a reboot without network	Did `sudo hwclock -w` run while the time was right? Coin cell in the clock?

Still stuck? Note what you see (shell output, LED, photo of the wiring) and post it in the team chat.